Name: Aditi Mittal
PRN: 23610067
Class: L.Y. CSE
Batch: B1

Experiment no 7
Title: MPI Collective Communication and Parallel Matrix Computations

1. Aim
To understand and implement MPI collective communication operations and apply them to parallel matrix computations using distributed-memory programming.

You will:

- Understand collective communication in MPI.
- Implement MPI_Bcast(), MPI_Scatter(), MPI_Gather(), and MPI_Reduce().
- Implement parallel Matrix–Vector Multiplication using MPI.
- Implement parallel Matrix–Matrix Multiplication using MPI.
- Apply different data decomposition strategies.
- Measure and analyze communication and computation performance.
- Understand the effect of the number of processes and decomposition strategy on Performance.

2. Why Are We Learning This?
In distributed-memory HPC systems, data is divided among multiple processes. Unlike OpenMP, where threads share a common memory, MPI processes need to exchange data explicitly.

For example, consider a large matrix:
[C = A* B]

If the matrix is too large for a single machine or the computation is time-consuming, different portions of the matrix can be assigned to different processes.

MPI collective operations provide efficient mechanisms to distribute and combine this data.

These concepts are fundamental to applications such as:

- Scientific and engineering simulations
- Computational fluid dynamics
- Weather and climate modelling
- Image and video processing
- Machine learning and AI
- Large-scale numerical linear algebra
- Distributed data analytics

3. HPC in Practice
Consider a weather simulation working with a large numerical grid.
The complete grid cannot efficiently be processed by one process. The grid can be divided among multiple processes:
Process 0 → Region 1
Process 1 → Region 2
Process 2 → Region 3
Process 3 → Region 4

MPI can then be used to:
1. Distribute input data.
2. Perform computation on local data.
3. Exchange required information.
4. Collect the final results.

This pattern of:
Distribute → Compute → Communicate → Combine
is one of the fundamental patterns in HPC programming.

4. Learning Focus

- Concept MPI Function / Technique
- Broadcast data MPI_Bcast()
- Distribute data MPI_Scatter()
- Collect data MPI_Gather()
- Combine results MPI_Reduce()
- Synchronization MPI_Barrier()
- Matrix–Vector Multiplication Row-wise decomposition
- Matrix–Matrix Multiplication Row-wise / block decomposition
- Performance Time, speedup, efficiency
- Concept MPI Function / Technique
- Advanced concept MPI_Scatterv()

5. Pre-Lab Activity
Before performing the assignment, answer the following:

1. What is collective communication in MPI?

Collective communication is communication in which a group of MPI processes participates in the same communication operation.

Unlike point-to-point communication, where one process communicates directly with another, collective operations involve **multiple processes within a communicator**.

Examples include:

- `MPI_Bcast()`
- `MPI_Scatter()`
- `MPI_Gather()`
- `MPI_Reduce()`
- `MPI_Barrier()`

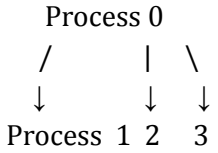
2. How is collective communication different from point-to-point communication?

Point-to-point communication	Collective communication
Usually involves two processes	Involves a group of processes
Uses operations such as `MPI_Send()` and `MPI_Recv()`	Uses operations such as `MPI_Bcast()` and `MPI_Scatter()`
Sender and receiver are explicitly specified	The participating processes are defined by the communicator
Useful for direct data exchange	Useful for distributing, collecting, or combining data among processes

Example:

Point-to-point:

Process 0 ——————→ Process 1
Collective:



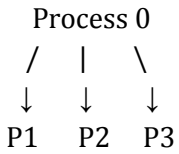
3. What is the purpose of MPI_Bcast()?

`MPI_Bcast()` is used to **broadcast the same data from one process to all other processes** in a communicator.

The process that initially has the data is called the **root process**.

For example:

Root



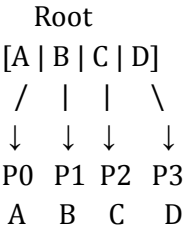
If Process 0 has a matrix size or configuration value that every process needs, it can use `MPI_Bcast()` to send that information to all processes.

4. What is the difference between MPI_Scatter() and MPI_Gather()?

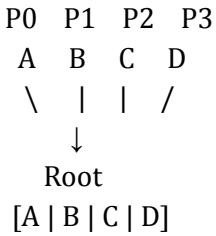
They perform opposite operations.

`MPI_Scatter()` distributes different portions of data from one root process to all processes.

Before:



`MPI_Gather()` collects data from all processes and brings it to a root process.



`MPI_Scatter()`	**`MPI_Gather()`**
Distributes data	Collects data
Root → all processes	All processes → root
Each process receives a portion	Root receives a portion from each process

5. What does MPI_Reduce() do?

`MPI_Reduce()` **combines values from multiple processes using an operation and places the final result at a root process**.

Common operations include:

- `MPI_SUM` → addition
- `MPI_MAX` → maximum value
- `MPI_MIN` → minimum value
- `MPI_PROD` → multiplication

For example:

P0 → 10
P1 → 20
P2 → 30
P3 → 40

MPI_Reduce(..., MPI_SUM, ...)

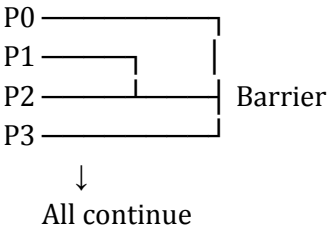
Result at root = 100

It is useful when each process calculates a **partial result** and the results need to be combined.

6. Why is MPI_Barrier() used?

`MPI_Barrier()` is used for **synchronization**.

When processes reach a barrier, they wait until **all processes in the communicator have reached that barrier**.



For example, it can be used when all processes must finish one stage of computation before starting the next stage.

However, barriers should not be used unnecessarily because waiting for slower processes can increase execution time.

7. What is data decomposition?

Data decomposition is the process of dividing a large dataset into smaller portions so that different processes can work on different portions simultaneously.

For example, a matrix can be divided by rows:

Original Matrix

[Row 0]
[Row 1]
[Row 2]
[Row 3]
[Row 4]
[Row 5]
[Row 6]
[Row 7]

With 4 processes:

P0 → Rows 0–1
P1 → Rows 2–3
P2 → Rows 4–5
P3 → Rows 6–7

Each process performs computation on its assigned data.

8. Why is matrix computation suitable for parallel processing?

Matrix computations are often suitable for parallel processing because many operations can be performed **independently on different elements or portions of a matrix**.

For example, in matrix addition:

Cij=Aij+BijC_{ij} = A_{ij} + B_{ij}

The calculation of one element does not normally depend on the calculation of another element.

Therefore:

P0 → Calculate part of C
P1 → Calculate another part of C
P2 → Calculate another part of C
P3 → Calculate remaining part of C

This allows multiple processes to perform calculations simultaneously and can reduce the overall execution time.

9. What is row-wise decomposition?

Row-wise decomposition is a method of dividing a matrix by assigning different rows to different processes.

For example, consider an 8-row matrix and 4 processes:

Matrix

```
┌──────────────────┐
│ Row 0            │ → P0
│ Row 1            │ → P0
│ Row 2            │ → P1
│ Row 3            │ → P1
│ Row 4            │ → P2
│ Row 5            │ → P2
│ Row 6            │ → P3
│ Row 7            │ → P3
└──────────────────┘
```

Each process works on its assigned rows.

Row-wise decomposition is simple to implement and is particularly useful when the computation can be performed independently on each row.

10. What happens if the number of matrix rows is not evenly divisible by the number of Processes?

If the number of rows is not evenly divisible by the number of processes, the rows **cannot be divided equally** among all processes.

For example:

- Matrix = **10 rows**
- Processes = **3**

The rows could be distributed as:

P0 → 4 rows
P1 → 3 rows
P2 → 3 rows

One process performs slightly more work than the others. This creates **load imbalance**.

MPI programs can handle this using techniques such as:

- Giving some processes one extra row
- Using `MPI_Scatterv()` for variable-sized data distribution
- Designing a more balanced decomposition

The goal is to keep the workload as evenly distributed as possible so that no process remains idle while another process is still computing.

Section A – Basics: MPI Collective Communication

Task 1: Broadcast Using MPI_Bcast()

Write an MPI program in which:
Process 0 initializes an integer value.
Process 0 broadcasts the value to all other processes using MPI_Bcast().
Every process displays the received value and its rank.
Suggested execution
Run the program using:
2 processes
4 processes
8 processes
Observe

Think
Why is broadcasting more appropriate than performing multiple individual MPI_Send() Operations?

Algorithm 1 — Broadcast

1. Initialize MPI.
2. Obtain process rank.
3. Process 0 initializes the value.
4. Call MPI_Bcast().
5. Every process displays the received value.
6. Finalize MPI.

```
#include <stdio.h>
#include <mpi.h>
```

```
int main(int argc, char *argv[])
{
   int rank;
   int value;

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);

   if (rank == 0)
   {
      value = 100;
   }

   MPI_Bcast(
      &value,
      1,
      MPI_INT,
      0,
      MPI_COMM_WORLD
   );

   printf(
      "Process %d received value = %d\n",
      rank,
      value
   );

   MPI_Finalize();

   return 0;
}
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 2 ./broadcast
Process 0 received value = 100
Process 1 received value = 100
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 4 ./broadcast
Process 0 received value = 100
Process 1 received value = 100
Process 2 received value = 100
Process 3 received value = 100
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 8 ./broadcast
Process 0 received value = 100
Process 1 received value = 100
Process 2 received value = 100
Process 3 received value = 100
Process 4 received value = 100
Process 5 received value = 100
Process 6 received value = 100
Process 7 received value = 100
```

Task 2: Scatter and Gather

Create an integer array of size 16 at Process 0.
Use:
MPI_Scatter()
to distribute the array among all processes.
Each process should:
1. Receive its portion.
2. Calculate the local sum.
3. Send the local result back using MPI_Gather().
Display the local sum calculated by each process and the final result at Process 0.
Example
For:
Array = [1,2,3,...,16]
with 4 processes:
Process 0 → [1,2,3,4]
Process 1 → [5,6,7,8]
Process 2 → [9,10,11,12]
Process 3 → [13,14,15,16]

Algorithm 2 — Scatter/Gather

1. Initialize MPI.
2. Process 0 initializes the array.
3. Scatter equal portions of the array.

4. Each process calculates its local sum.
5. Gather the local sums at Process 0.
6. Process 0 calculates the final sum.
7. Finalize MPI.

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
   int rank, size;

   int array[16];
   int local_array[16];
   int local_sum;

   int gathered_sums[16];

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   if (size != 4)
   {
      if (rank == 0)
      {
         printf("Please run this program with 4 processes.\n");
      }

      MPI_Finalize();
      return 0;
   }

   if (rank == 0)
   {
      for (int i = 0; i < 16; i++)
      {
         array[i] = i + 1;
      }
   }

   MPI_Scatter(
      array,
      4,
      MPI_INT,
```

```
    local_array,
    4,
    MPI_INT,
    0,
    MPI_COMM_WORLD
  );

  local_sum = 0;

  for (int i = 0; i < 4; i++)
  {
    local_sum += local_array[i];
  }

  printf(
    "Process %d local sum = %d\n",
    rank,
    local_sum
  );

  MPI_Gather(
    &local_sum,
    1,
    MPI_INT,
    gathered_sums,
    1,
    MPI_INT,
    0,
    MPI_COMM_WORLD
  );

  if (rank == 0)
  {
    int global_sum = 0;

    for (int i = 0; i < 4; i++)
    {
      global_sum += gathered_sums[i];
    }

    printf("Global sum = %d\n", global_sum);
  }

  MPI_Finalize();

  return 0;
}
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpicc Task2/sum.c -o sum
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 2 ./sum
Please run this program with 4 processes.
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 4 ./sum
Process 0 local sum = 10
Global sum = 136
Process 1 local sum = 26
Process 2 local sum = 42
Process 3 local sum = 58
```

Task 3: Global Reduction Using MPI_Reduce()

Modify Task 2 to use:
MPI_Reduce()
to calculate the global sum.
Also calculate:
Global maximum
Global minimum
using appropriate MPI reduction operations.
Compare the use of MPI_Gather() and MPI_Reduce().

Discussion
Why is MPI_Reduce() preferable when only the final aggregate value is required?

Algorithm 3 — Reduce

1. Initialize MPI.
2. Scatter the input data.
3. Calculate local sum, minimum and maximum.
4. Use MPI_Reduce() with MPI_SUM.
5. Use MPI_Reduce() with MPI_MIN.
6. Use MPI_Reduce() with MPI_MAX.
7. Process 0 displays the global results.
8. Finalize MPI.

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
   int rank, size;
```

```
int array[16];
int local_array[16];

int local_sum;
int local_max;
int local_min;

int global_sum;
int global_max;
int global_min;

MPI_Init(&argc, &argv);

MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

if (size != 4)
{
   if (rank == 0)
   {
      printf("Please run this program with 4 processes.\n");
   }

   MPI_Finalize();
   return 0;
}

if (rank == 0)
{
   for (int i = 0; i < 16; i++)
   {
      array[i] = i + 1;
   }
}

MPI_Scatter(
   array,
   4,
   MPI_INT,
   local_array,
   4,
   MPI_INT,
   0,
   MPI_COMM_WORLD
);

local_sum = 0;
local_max = local_array[0];
```

```
local_min = local_array[0];

for (int i = 0; i < 4; i++)
{
   local_sum += local_array[i];

   if (local_array[i] > local_max)
      local_max = local_array[i];

   if (local_array[i] < local_min)
      local_min = local_array[i];
}

MPI_Reduce(
   &local_sum,
   &global_sum,
   1,
   MPI_INT,
   MPI_SUM,
   0,
   MPI_COMM_WORLD
);

MPI_Reduce(
   &local_max,
   &global_max,
   1,
   MPI_INT,
   MPI_MAX,
   0,
   MPI_COMM_WORLD
);

MPI_Reduce(
   &local_min,
   &global_min,
   1,
   MPI_INT,
   MPI_MIN,
   0,
   MPI_COMM_WORLD
);

if (rank == 0)
{
   printf("Global Sum = %d\n", global_sum);
   printf("Global Maximum = %d\n", global_max);
   printf("Global Minimum = %d\n", global_min);
}
```

```
    MPI_Finalize();

    return 0;
}
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpicc Task3/mpi_reduce.c -
o mpi_reduce
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 4 ./mpi_reduce
Global Sum = 136
Global Maximum = 16
Global Minimum = 1
```

Section B – Moderate Application: Matrix Computations

Task 4: MPI Matrix–Vector Multiplication

Implement:
[Y = A X]
where:
(A) is an (N N) matrix.
(X) is an (N) vector.
(Y) is an (N) result vector.
Step 1 – Sequential Version
Implement the matrix-vector multiplication sequentially.
Measure its execution time.
Step 2 – MPI Version
Implement the MPI version using row-wise decomposition.
Vector X → Broadcast to all processes
Use:
MPI_Scatter() to distribute matrix rows.
MPI_Bcast() to distribute the vector.
Local computation at each process.
MPI_Gather() to collect the result.

Algorithm 4 — Matrix–Vector

1. Initialize MPI.
2. Process 0 initializes A and X.
3. Broadcast X to all processes.
4. Scatter rows of A.
5. Each process computes its local Y.

6. Gather local Y values.
7. Process 0 obtains the complete Y.
8. Finalize MPI.

Sequential:

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main()
{
   int N = 500;

   double *A = malloc(N * N * sizeof(double));
   double *X = malloc(N * sizeof(double));
   double *Y = malloc(N * sizeof(double));

   for (int i = 0; i < N; i++)
   {
      X[i] = 1.0;

      for (int j = 0; j < N; j++)
      {
         A[i * N + j] = 1.0;
      }
   }

   double start = MPI_Wtime();

   for (int i = 0; i < N; i++)
   {
      Y[i] = 0.0;

      for (int j = 0; j < N; j++)
      {
         Y[i] += A[i * N + j] * X[j];
      }
   }

   double end = MPI_Wtime();

   printf(
      "Matrix Size = %d x %d\n",
      N,
      N
```

```
    );

    printf(
        "Sequential Time = %.6f seconds\n",
        end - start
    );

    free(A);
    free(X);
    free(Y);

    return 0;
}
```

MPI:

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
    int rank, size;
    int N = 8;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (N % size != 0)
    {
        if (rank == 0)
        {
            printf("N must be divisible by number of processes.\n");
        }

        MPI_Finalize();
        return 0;
    }

    int rows_per_process = N / size;

    double *A = NULL;
    double *X = malloc(N * sizeof(double));
    double *Y = NULL;
```

```
double *local_A =
   malloc(rows_per_process * N * sizeof(double));

double *local_Y =
   malloc(rows_per_process * sizeof(double));

if (rank == 0)
{
   A = malloc(N * N * sizeof(double));
   Y = malloc(N * sizeof(double));

   for (int i = 0; i < N; i++)
   {
      X[i] = 1.0;

      for (int j = 0; j < N; j++)
      {
         A[i * N + j] = 1.0;
      }
   }
}

MPI_Bcast(
   X,
   N,
   MPI_DOUBLE,
   0,
   MPI_COMM_WORLD
);

double start = MPI_Wtime();

MPI_Scatter(
   A,
   rows_per_process * N,
   MPI_DOUBLE,
   local_A,
   rows_per_process * N,
   MPI_DOUBLE,
   0,
   MPI_COMM_WORLD
);

for (int i = 0; i < rows_per_process; i++)
{
   local_Y[i] = 0.0;

   for (int j = 0; j < N; j++)
```

```
    {
      local_Y[i] +=
        local_A[i * N + j] * X[j];
    }
  }

  MPI_Gather(
    local_Y,
    rows_per_process,
    MPI_DOUBLE,
    Y,
    rows_per_process,
    MPI_DOUBLE,
    0,
    MPI_COMM_WORLD
  );

  double end = MPI_Wtime();

  if (rank == 0)
  {
    printf("Matrix Size = %d x %d\n", N, N);
    printf("MPI Processes = %d\n", size);
    printf("MPI Time = %.6f seconds\n", end - start);

    printf("Result vector:\n");

    for (int i = 0; i < N; i++)
    {
      printf("%.2f ", Y[i]);
    }

    printf("\n");
  }

  free(X);
  free(local_A);
  free(local_Y);

  if (rank == 0)
  {
    free(A);
    free(Y);
  }

  MPI_Finalize();

  return 0;
}
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpicc Task4/matrix-vector-mpi.c
 -o matrix-vector-mpi
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 4 ./matrix-vector-mp
i
Matrix Size = 8 x 8
MPI Processes = 4
MPI Time = 0.000048 seconds
Result vector:
8.00 8.00 8.00 8.00 8.00 8.00 8.00 8.00
```

Performance Analysis

Test with different matrix sizes, for example:

500 × 500

1000 × 1000

2000 × 2000

and different numbers of processes:

2

4

8

Record:

Matrix

Size Processes Sequential Time MPI Time Speedup Efficiency

500 2

500 4

500 8

1000 2

1000 4

1000 8

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ gcc -O2 Task4/matrix-vector-seq
.c -o matrix-vector-seq
mpicc -O2 Task4/matrix-vector-mpi.c -o matrix-vector-mpi
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ ./matrix-vector-seq 500
Matrix Size = 500 x 500
Sequential Time = 0.000145 seconds
Result checksum = 250000.00
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 2 ./matrix-vector-mp
i 500
Matrix Size = 500 x 500
MPI Processes = 2
MPI Time = 0.000851 seconds
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 4 ./matrix-vector-mp
i 500
Matrix Size = 500 x 500
MPI Processes = 4
MPI Time = 0.000932 seconds
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 8 ./matrix-vector-mp
i 500
Matrix Size = 500 x 500
MPI Processes = 8
MPI Time = 0.000604 seconds
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ ./matrix-vector-seq 1000
Matrix Size = 1000 x 1000
Sequential Time = 0.001093 seconds
Result checksum = 1000000.00
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 2 ./matrix-vector-mp
i 1000
Matrix Size = 1000 x 1000
MPI Processes = 2
MPI Time = 0.003187 seconds
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 4 ./matrix-vector-mp
i 1000
Matrix Size = 1000 x 1000
MPI Processes = 4
MPI Time = 0.002298 seconds
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 8 ./matrix-vector-mp
i 1000
Matrix Size = 1000 x 1000
MPI Processes = 8
MPI Time = 0.002477 seconds
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ ./matrix-vector-seq 2000
Matrix Size = 2000 x 2000
Sequential Time = 0.002744 seconds
Result checksum = 4000000.00
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 2 ./matrix-vector-mp
i 2000
Matrix Size = 2000 x 2000
MPI Processes = 2
MPI Time = 0.020924 seconds
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 4 ./matrix-vector-mp
i 2000
Matrix Size = 2000 x 2000
MPI Processes = 4
MPI Time = 0.011608 seconds
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 8 ./matrix-vector-mp
i 2000
Matrix Size = 2000 x 2000
MPI Processes = 8
MPI Time = 0.012969 seconds
```

Task 5: MPI Matrix–Matrix Multiplication

Implement:
[C = A * B]
for two square matrices (A) and (B).

Sequential Version
First implement the sequential algorithm.

MPI Version
Implement a parallel version using row-wise decomposition.

Suggested approach:

1. Process 0 initializes matrices (A) and (B).
2. Distribute rows of (A) using MPI_Scatter().
3. Broadcast matrix (B) using MPI_Bcast().
4. Each process computes its assigned rows of (C).
5. Collect the rows of (C) using MPI_Gather().

Algorithm 5 — Matrix–Matrix

1. Initialize MPI.
2. Process 0 initializes A and B.
3. Scatter rows of A.
4. Broadcast B.
5. Each process computes its assigned rows of C.
6. Gather rows of C.
7. Process 0 obtains complete matrix C.
8. Finalize MPI.

Matrix-Matrix multiplication: Sequential

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
   int N = 500;

   double *A = malloc(N * N * sizeof(double));
   double *B = malloc(N * N * sizeof(double));
   double *C = malloc(N * N * sizeof(double));

   // Initialize matrices
   for (int i = 0; i < N; i++)
   {
      for (int j = 0; j < N; j++)
      {
         A[i * N + j] = 1.0;
         B[i * N + j] = 1.0;
         C[i * N + j] = 0.0;
      }
   }

   clock_t start = clock();
```

```
   // Matrix multiplication: C = A * B
   for (int i = 0; i < N; i++)
   {
      for (int j = 0; j < N; j++)
      {
         C[i * N + j] = 0.0;

         for (int k = 0; k < N; k++)
         {
            C[i * N + j] +=
               A[i * N + k] * B[k * N + j];
         }
      }
   }

   clock_t end = clock();

   double time =
      (double)(end - start) / CLOCKS_PER_SEC;

   printf("Matrix Size = %d x %d\n", N, N);
   printf("Sequential Time = %.6f seconds\n", time);

   printf("Sample Result C[0][0] = %.2f\n", C[0]);

   free(A);
   free(B);
   free(C);

   return 0;
}
```

Matrix-Matrix Multiplication MPI

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
   int rank, size;

   int N = 8;

   MPI_Init(&argc, &argv);
```

```
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

if (N % size != 0)
{
   if (rank == 0)
   {
      printf(
         "Matrix size must be divisible by number of processes.\n"
      );
   }

   MPI_Finalize();
   return 0;
}

int rows_per_process = N / size;

double *A = NULL;
double *B = NULL;
double *C = NULL;

double *local_A =
   malloc(rows_per_process * N * sizeof(double));

double *local_C =
   malloc(rows_per_process * N * sizeof(double));

/*
 * B is required by every process,
 * so every process allocates memory for B.
 */
B = malloc(N * N * sizeof(double));

if (rank == 0)
{
   A = malloc(N * N * sizeof(double));
   C = malloc(N * N * sizeof(double));

   // Initialize A and B
   for (int i = 0; i < N; i++)
   {
      for (int j = 0; j < N; j++)
      {
         A[i * N + j] = 1.0;
         B[i * N + j] = 1.0;
      }
   }
}
```

```
/*
 * Broadcast matrix B to all processes.
 */
MPI_Bcast(
   B,
   N * N,
   MPI_DOUBLE,
   0,
   MPI_COMM_WORLD
);

MPI_Barrier(MPI_COMM_WORLD);

double start = MPI_Wtime();

/*
 * Distribute rows of A.
 */
MPI_Scatter(
   A,
   rows_per_process * N,
   MPI_DOUBLE,
   local_A,
   rows_per_process * N,
   MPI_DOUBLE,
   0,
   MPI_COMM_WORLD
);

/*
 * Local matrix multiplication.
 *
 * local_C = local_A * B
 */
for (int i = 0; i < rows_per_process; i++)
{
   for (int j = 0; j < N; j++)
   {
      local_C[i * N + j] = 0.0;

      for (int k = 0; k < N; k++)
      {
         local_C[i * N + j] +=
            local_A[i * N + k] *
            B[k * N + j];
      }
   }
}
```

```
/*
 * Collect all rows of C at Process 0.
 */
MPI_Gather(
   local_C,
   rows_per_process * N,
   MPI_DOUBLE,
   C,
   rows_per_process * N,
   MPI_DOUBLE,
   0,
   MPI_COMM_WORLD
);

MPI_Barrier(MPI_COMM_WORLD);

double end = MPI_Wtime();

if (rank == 0)
{
   printf("Matrix Size = %d x %d\n", N, N);
   printf("MPI Processes = %d\n", size);
   printf("MPI Time = %.6f seconds\n",
         end - start);

   printf("Sample Result C[0][0] = %.2f\n",
         C[0]);

   printf("Result Matrix:\n");

   for (int i = 0; i < N; i++)
   {
      for (int j = 0; j < N; j++)
      {
         printf("%.2f ", C[i * N + j]);
      }

      printf("\n");
   }
}

free(B);
free(local_A);
free(local_C);

if (rank == 0)
{
   free(A);
```

```
    free(C);
  }

  MPI_Finalize();

  return 0;
}
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ gcc -O2 Task5/matrix-matrix-seq
.c -o matrix-matrix-seq
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpicc -O2 Task5/matrix-matrix-m
pi.c -o matrix-matrix-mpi
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ ./matrix-matrix-seq 500
Matrix Size = 500 x 500
Sequential Time = 0.043197 seconds
Sample Result C[0][0] = 500.00
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 2 ./matrix-matrix-mp
i 500
Matrix Size = 8 x 8
MPI Processes = 2
MPI Time = 0.000026 seconds
Sample Result C[0][0] = 8.00
Result Matrix:
8.00 8.00 8.00 8.00 8.00 8.00 8.00 8.00
8.00 8.00 8.00 8.00 8.00 8.00 8.00 8.00
8.00 8.00 8.00 8.00 8.00 8.00 8.00 8.00
8.00 8.00 8.00 8.00 8.00 8.00 8.00 8.00
8.00 8.00 8.00 8.00 8.00 8.00 8.00 8.00
8.00 8.00 8.00 8.00 8.00 8.00 8.00 8.00
8.00 8.00 8.00 8.00 8.00 8.00 8.00 8.00
8.00 8.00 8.00 8.00 8.00 8.00 8.00 8.00
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 4 ./matrix-matrix-mp
i 500
Matrix Size = 8 x 8
MPI Processes = 4
MPI Time = 0.000045 seconds
Sample Result C[0][0] = 8.00
Result Matrix:
8.00 8.00 8.00 8.00 8.00 8.00 8.00 8.00
8.00 8.00 8.00 8.00 8.00 8.00 8.00 8.00
8.00 8.00 8.00 8.00 8.00 8.00 8.00 8.00
8.00 8.00 8.00 8.00 8.00 8.00 8.00 8.00
8.00 8.00 8.00 8.00 8.00 8.00 8.00 8.00
8.00 8.00 8.00 8.00 8.00 8.00 8.00 8.00
8.00 8.00 8.00 8.00 8.00 8.00 8.00 8.00
8.00 8.00 8.00 8.00 8.00 8.00 8.00 8.00
```

Section C – Challenge: Decomposition and Optimization

Task 6: Compare Different Matrix Decomposition Strategies

Modify the MPI matrix-matrix multiplication program to investigate different decomposition strategies.

Study:

1. Row-wise decomposition

Each process receives a set of rows of matrix (A).

2. Column-wise decomposition

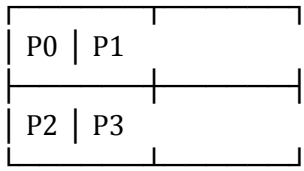
Each process is responsible for a set of columns of matrix (B).

3. Block decomposition

Divide the matrix into smaller blocks.

For example:

Matrix



Investigate how the decomposition affects:

- **Amount of data communicated.**
- **Computation performed by each process.**
- **Memory usage.**
- **Load balancing.**
- **Execution time.**
- **Scalability.**

Column Decomposition:

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
   int rank, size;

   int N = 8;

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   if (N % size != 0)
   {
```

```
    if (rank == 0)
    {
        printf(
            "Matrix size must be divisible by number of processes.\n"
        );
    }

    MPI_Finalize();
    return 0;
}

int cols_per_process = N / size;

double *A = NULL;
double *B = NULL;
double *C = NULL;

double *local_B =
    malloc(N * cols_per_process * sizeof(double));

double *local_C =
    malloc(N * cols_per_process * sizeof(double));

/*
 * Every process needs A.
 */
A = malloc(N * N * sizeof(double));

if (rank == 0)
{
    B = malloc(N * N * sizeof(double));
    C = malloc(N * N * sizeof(double));

    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            A[i * N + j] = 1.0;
            B[i * N + j] = 1.0;
        }
    }
}
else
{
    B = malloc(N * N * sizeof(double));
}

/*
 * Broadcast A to every process.
```

```
 */
MPI_Bcast(
   A,
   N * N,
   MPI_DOUBLE,
   0,
   MPI_COMM_WORLD
);

/*
 * Distribute columns of B.
 *
 * B is stored row-wise, so MPI_Scatter
 * cannot directly send contiguous columns.
 *
 * Therefore, Process 0 creates a temporary
 * column-wise representation.
 */

double *B_transposed = NULL;

if (rank == 0)
{
   B_transposed =
      malloc(N * N * sizeof(double));

   for (int i = 0; i < N; i++)
   {
      for (int j = 0; j < N; j++)
      {
         B_transposed[j * N + i] =
            B[i * N + j];
      }
   }
}

/*
 * Scatter columns of B after transposing it.
 */
MPI_Scatter(
   B_transposed,
   cols_per_process * N,
   MPI_DOUBLE,
   local_B,
   cols_per_process * N,
   MPI_DOUBLE,
   0,
   MPI_COMM_WORLD
);
```

```
/*
 * Each process computes its assigned
 * columns of C.
 */
for (int i = 0; i < N; i++)
{
   for (int j = 0; j < cols_per_process; j++)
   {
      local_C[i * cols_per_process + j] = 0.0;

      for (int k = 0; k < N; k++)
      {
         local_C[i * cols_per_process + j] +=
            A[i * N + k] *
            local_B[j * N + k];
      }
   }
}

/*
 * Gather the local column blocks.
 */
double *C_transposed = NULL;

if (rank == 0)
{
   C_transposed =
      malloc(N * N * sizeof(double));
}

MPI_Gather(
   local_C,
   N * cols_per_process,
   MPI_DOUBLE,
   C_transposed,
   N * cols_per_process,
   MPI_DOUBLE,
   0,
   MPI_COMM_WORLD
);

if (rank == 0)
{
   /*
    * Convert gathered column representation
    * back to normal row-major C.
    */
   for (int p = 0; p < size; p++)
```

```
    {
      for (int i = 0; i < N; i++)
      {
        for (int j = 0; j < cols_per_process; j++)
        {
          int global_col =
            p * cols_per_process + j;

          C[i * N + global_col] =
            C_transposed[
              p * N * cols_per_process +
              i * cols_per_process +
              j
            ];
        }
      }
    }

    printf("Column-wise Matrix Multiplication\n");
    printf("Matrix Size = %d x %d\n", N, N);
    printf("MPI Processes = %d\n", size);
    printf("C[0][0] = %.2f\n", C[0]);

    free(B_transposed);
    free(C_transposed);
    free(C);
  }

  free(A);
  free(B);
  free(local_B);
  free(local_C);

  MPI_Finalize();

  return 0;
}
```

Block Decomposition:

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
  int rank, size;
```

```
int N = 8;

MPI_Init(&argc, &argv);

MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

/*
 * This implementation uses 4 processes:
 *
 * P0 P1
 * P2 P3
 */
if (size != 4)
{
   if (rank == 0)
   {
      printf(
         "Please run this program with 4 processes.\n"
      );
   }

   MPI_Finalize();
   return 0;
}

int block_size = N / 2;

double *A = NULL;
double *B = NULL;
double *C = NULL;

double *local_A =
   malloc(block_size * N * sizeof(double));

double *local_B =
   malloc(N * block_size * sizeof(double));

double *local_C =
   calloc(block_size * block_size,
         sizeof(double));

if (rank == 0)
{
   A = malloc(N * N * sizeof(double));
   B = malloc(N * N * sizeof(double));
   C = malloc(N * N * sizeof(double));
```

```
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            A[i * N + j] = 1.0;
            B[i * N + j] = 1.0;
            C[i * N + j] = 0.0;
        }
    }
}

/*
 * For this simple demonstration:
 *
 * Every process receives the required
 * portions of A and B.
 *
 * A complete implementation for arbitrary
 * process grids would use MPI Cartesian
 * communicators and subarray datatypes.
 */

MPI_Bcast(
    B,
    N * N,
    MPI_DOUBLE,
    0,
    MPI_COMM_WORLD
);

/*
 * Broadcast A as well.
 */
MPI_Bcast(
    A,
    N * N,
    MPI_DOUBLE,
    0,
    MPI_COMM_WORLD
);

/*
 * Determine block position.
 *
 * P0 -> block (0,0)
 * P1 -> block (0,1)
 * P2 -> block (1,0)
 * P3 -> block (1,1)
 */
```

```
int block_row = rank / 2;
int block_col = rank % 2;

/*
 * Each process computes its C block.
 *
 * For a block C(i,j), we calculate:
 *
 * C(i,j) = sum A(i,k) * B(k,j)
 */
for (int i = 0; i < block_size; i++)
{
   for (int j = 0; j < block_size; j++)
   {
      double sum = 0.0;

      for (int k = 0; k < N; k++)
      {
         int global_i =
            block_row * block_size + i;

         int global_j =
            block_col * block_size + j;

         sum +=
            A[global_i * N + k] *
            B[k * N + global_j];
      }

      local_C[i * block_size + j] = sum;
   }
}

/*
 * Gather blocks at Process 0.
 */
double *all_blocks = NULL;

if (rank == 0)
{
   all_blocks =
      malloc(N * N * sizeof(double));
}

MPI_Gather(
   local_C,
   block_size * block_size,
   MPI_DOUBLE,
   all_blocks,
```

```
    block_size * block_size,
    MPI_DOUBLE,
    0,
    MPI_COMM_WORLD
);

if (rank == 0)
{
    /*
     * Rearrange blocks into C.
     */
    for (int p = 0; p < 4; p++)
    {
        int p_row = p / 2;
        int p_col = p % 2;

        for (int i = 0; i < block_size; i++)
        {
            for (int j = 0; j < block_size; j++)
            {
                int global_i =
                    p_row * block_size + i;

                int global_j =
                    p_col * block_size + j;

                C[global_i * N + global_j] =
                    all_blocks[
                        p * block_size * block_size +
                        i * block_size +
                        j
                    ];
            }
        }
    }

    printf("Block-wise Matrix Multiplication\n");
    printf("Matrix Size = %d x %d\n", N, N);
    printf("MPI Processes = %d\n", size);
    printf("C[0][0] = %.2f\n", C[0]);

    free(A);
    free(B);
    free(C);
    free(all_blocks);
}

/*
 * Only non-root processes have NULL A/C,
```

```
   * but B was broadcast into memory allocated
   * below.
   */

  if (rank != 0)
  {
    free(B);
  }

  free(local_A);
  free(local_B);
  free(local_C);

  MPI_Finalize();

  return 0;
}
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpicc -O2 Task6/block-matrix-mu
l.c -o block-matrix-mul
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpicc -O2 Task6/column-matrix-m
ul.c -o column-matrix-mul
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpicc Task6/block-matrix-mul.c
-o block-matrix-mul
mpirun -np 4 ./block-matrix-mul 500
Block-wise Matrix Multiplication
Matrix Size = 500 x 500
MPI Processes = 4
C[0][0] = 500.00
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpicc Task6/column-matrix-mul.c
 -o column-matrix-mul
mpirun -np 4 ./column-matrix-mul 500
Column-wise Matrix Multiplication
Matrix Size = 8 x 8
MPI Processes = 4
C[0][0] = 8.00
```

Task 7: Non-Uniform Data Distribution Using MPI_Scatterv()

Consider a situation where the matrix cannot be divided equally among the processes.
For example:
N = 1 0
Number of processes = 3
The rows cannot be divided equally.
Investigate and implement matrix-vector multiplication using:
MPI_Scatterv()
instead of MPI_Scatter().

The distribution could be:
P0 → 4 rows
P1 → 3 rows
P2 → 3 rows

Analyze
Compare:
MPI_Scatter()
vs.
MPI_Scatterv()
in terms of:

- **Data distribution**
- **Flexibility**
- **Programming complexity**
- **Applicability to real HPC problems**

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
   int rank, size;

   int N = 10;

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   if (size != 3)
   {
      if (rank == 0)
      {
         printf(
            "Please run this program with 3 processes.\n"
         );
      }

      MPI_Finalize();
      return 0;
   }

   /*
```

```
 * Number of rows assigned to each process.
 *
 * 10 rows:
 *
 * P0 -> 4 rows
 * P1 -> 3 rows
 * P2 -> 3 rows
 */
int rows_per_process[3] = {4, 3, 3};

/*
 * Number of matrix elements sent to
 * each process.
 *
 * Each row contains N = 10 elements.
 */
int sendcounts[3] =
{
   4 * N,
   3 * N,
   3 * N
};

/*
 * Starting positions in flattened matrix.
 */
int displacements[3] =
{
   0,
   4 * N,
   7 * N
};

int local_rows =
   rows_per_process[rank];

double *A = NULL;
double *X = NULL;
double *Y = NULL;

double *local_A =
   malloc(local_rows * N * sizeof(double));

double *local_Y =
   malloc(local_rows * sizeof(double));

/*
 * Every process needs the complete vector X.
 */
```

```
X = malloc(N * sizeof(double));

if (rank == 0)
{
   A = malloc(N * N * sizeof(double));
   Y = malloc(N * sizeof(double));

   /*
    * Initialize A and X.
    */
   for (int i = 0; i < N; i++)
   {
      X[i] = 1.0;

      for (int j = 0; j < N; j++)
      {
         A[i * N + j] = 1.0;
      }
   }
}

/*
 * Broadcast vector X.
 */
MPI_Bcast(
   X,
   N,
   MPI_DOUBLE,
   0,
   MPI_COMM_WORLD
);

/*
 * Scatter unequal numbers of matrix elements.
 */
MPI_Scatterv(
   A,
   sendcounts,
   displacements,
   MPI_DOUBLE,
   local_A,
   local_rows * N,
   MPI_DOUBLE,
   0,
   MPI_COMM_WORLD
);

/*
 * Local matrix-vector multiplication.
```

```
 */
for (int i = 0; i < local_rows; i++)
{
   local_Y[i] = 0.0;

   for (int j = 0; j < N; j++)
   {
      local_Y[i] +=
         local_A[i * N + j] * X[j];
   }
}

/*
 * Since the number of result elements is
 * different for each process, use MPI_Gatherv().
 */
int recvcounts[3] =
{
   4,
   3,
   3
};

int recvdisplacements[3] =
{
   0,
   4,
   7
};

MPI_Gatherv(
   local_Y,
   local_rows,
   MPI_DOUBLE,
   Y,
   recvcounts,
   recvdisplacements,
   MPI_DOUBLE,
   0,
   MPI_COMM_WORLD
);

if (rank == 0)
{
   printf("MPI_Scatterv Matrix-Vector Multiplication\n");
   printf("Matrix Size = %d x %d\n", N, N);
   printf("MPI Processes = %d\n", size);

   printf("Row Distribution:\n");
```

```
    printf("Process 0 -> 4 rows\n");
    printf("Process 1 -> 3 rows\n");
    printf("Process 2 -> 3 rows\n");

    printf("\nResult Vector:\n");

    for (int i = 0; i < N; i++)
    {
       printf("%.2f ", Y[i]);
    }

    printf("\n");
  }

  free(X);
  free(local_A);
  free(local_Y);

  if (rank == 0)
  {
    free(A);
    free(Y);
  }

  MPI_Finalize();

  return 0;
}
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpicc Task7/scatter_v.c -o scat
ter_v
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A7$ mpirun -np 3 ./scatter_v
MPI_Scatterv Matrix-Vector Multiplication
Matrix Size = 10 x 10
MPI Processes = 3
Row Distribution:
Process 0 -> 4 rows
Process 1 -> 3 rows
Process 2 -> 3 rows

Result Vector:
10.00 10.00 10.00 10.00 10.00 10.00 10.00 10.00 10.00 10.00
```

6. Performance Analysis

For your MPI programs, measure:
Execution Time
Speedup

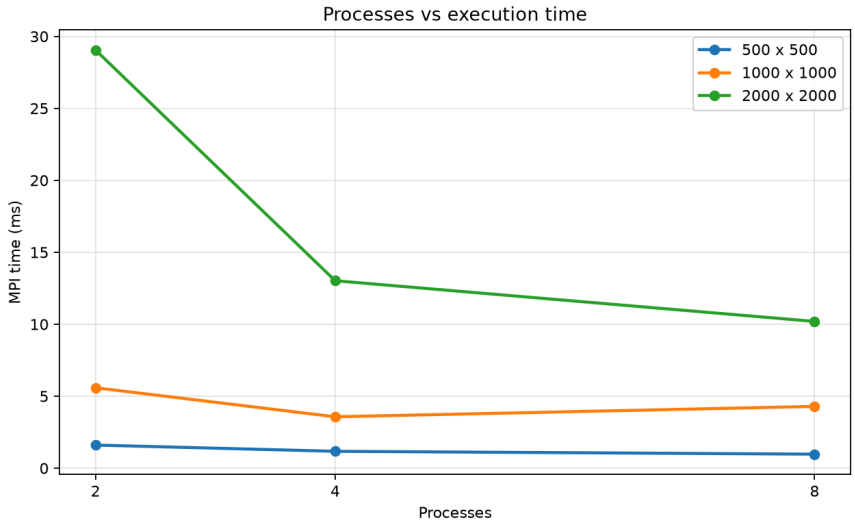
Efficiency
where (p) is the number of processes.

N x N	Processes	Sequential (ms)	MPI (ms)	Speedup	Efficiency (%)
500 x 500	2	0.202	1.606	0.13	6.3
500 x 500	4	0.202	1.173	0.17	4.3
500 x 500	8	0.202	0.974	0.21	2.6
1000 x 1000	2	0.966	5.587	0.17	8.6
1000 x 1000	4	0.966	3.572	0.27	6.8
1000 x 1000	8	0.966	4.292	0.23	2.8
2000 x 2000	2	3.611	29.052	0.12	6.2
2000 x 2000	4	3.611	13.024	0.28	6.9
2000 x 2000	8	3.611	10.196	0.35	4.4

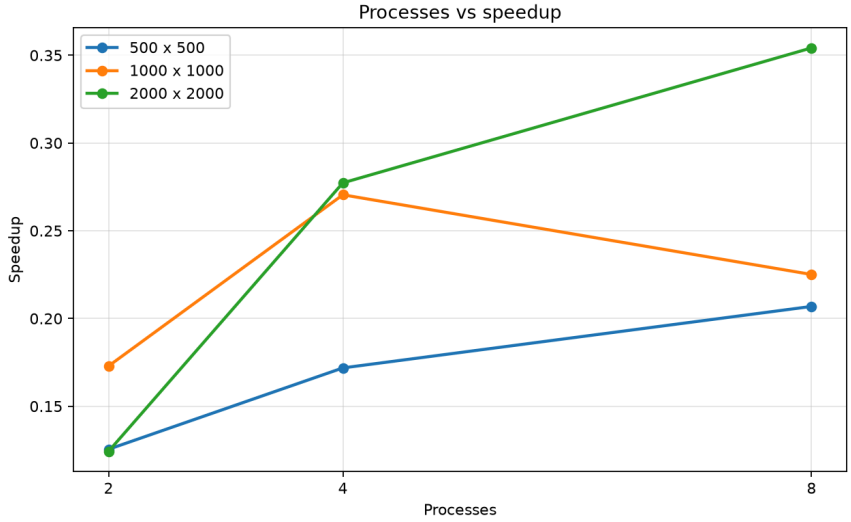
Also investigate:

- **Communication overhead**
- **Computation time**
- **Load imbalance**
- **Scalability**
- **Effect of matrix size**
- **Effect of number of processes**
- **Required Graphs**

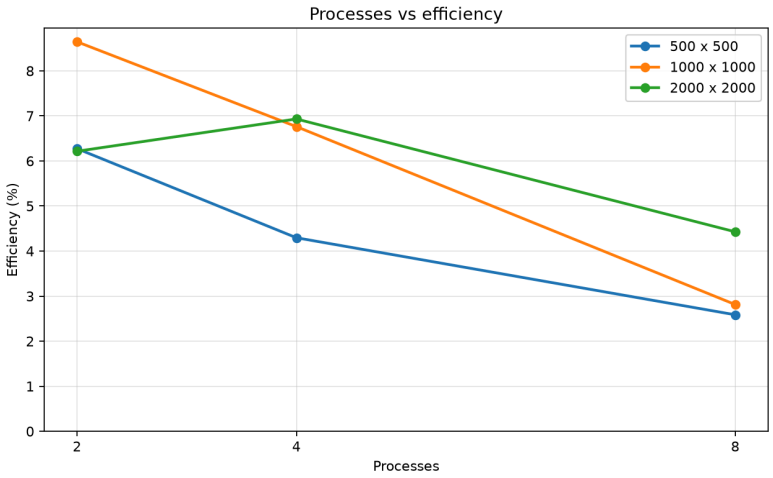
Prepare:
1. Number of Processes vs Execution Time



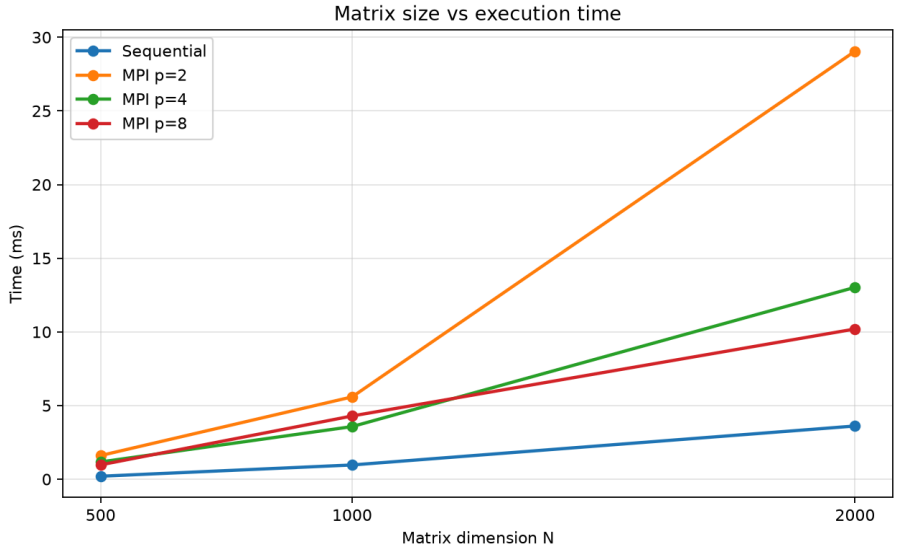
2. Number of Processes vs Speedup



3. Number of Processes vs Efficiency



4. Matrix Size vs Execution Time



N	p	Broadcast	Scatter	Compute min	Compute max	Gather
500	2	0.037	1.458	0.086	0.101	0.023
500	4	0.064	1.068	0.042	0.046	0.013
500	8	0.147	0.775	0.028	0.090	0.110
1000	2	0.267	4.839	0.421	0.433	0.052
1000	4	0.373	2.992	0.189	0.204	0.023
1000	8	1.192	3.110	0.119	0.199	0.079
2000	2	0.344	26.619	1.846	1.999	0.247
2000	4	0.611	11.380	0.857	1.038	0.054
2000	8	0.843	9.002	0.474	0.660	0.128

7. AI/HPC Case Study

Select one real-world application where distributed matrix computation or collective communication is used.

Suggested areas:

- **Deep learning**
- **Computer vision**
- **Scientific computing**
- **Weather forecasting**
- **Computational fluid dynamics**

- **Medical imaging**
- **Recommendation systems**
- **Large-scale data analytics**
- **Molecular simulation**

Write 300–500 words covering:

1. Problem being solved.
2. Why the computation is computationally intensive.
3. How data can be distributed.
4. Role of MPI communication.
5. Possible decomposition strategy.
6. Benefits of parallel execution.
7. Communication or scalability challenges.

AI/HPC Case Study: Distributed Matrix Computation in Deep Learning

Introduction

Deep learning is a major application of distributed matrix computation and parallel computing. Modern deep learning models, especially large neural networks used for image recognition, natural language processing, and generative AI, perform a huge number of mathematical operations on large datasets. Distributed computing allows these operations to be performed simultaneously across multiple processors or computing nodes.

1. Problem Being Solved

The goal of deep learning is to train a neural network using a large dataset so that it can learn patterns and make predictions. During training, the model repeatedly performs **matrix multiplications, additions, and other numerical operations** during the forward and backward passes.

2. Why is the Computation Computationally Intensive?

Neural networks can contain millions or billions of parameters. Training them requires processing large batches of data through multiple layers repeatedly. Matrix multiplication is one of the most frequently performed operations and can require billions of arithmetic operations during training.

For example, a layer can perform an operation such as:

$Y=XW+BY = XW + B$

where X represents input data, W represents model weights, and B represents bias values. Large dimensions of these matrices make the computation expensive.

3. How Can Data Be Distributed?

The training data can be divided into smaller batches and distributed among different processes. Each MPI process can receive a different portion of the training data and perform computations on its assigned batch.

For example:

Dataset
 |
 ├── Process 0 → Batch 1
 ├── Process 1 → Batch 2
 ├── Process 2 → Batch 3
 └── Process 3 → Batch 4

4. Role of MPI Communication

MPI can coordinate the processes and exchange intermediate results. After processing their individual batches, processes may need to **combine their computed gradients** so that the model parameters can be updated consistently. Collective operations such as `MPI_Reduce()` or `MPI_Allreduce()` can be used for this purpose.

5. Possible Decomposition Strategy

A common approach is **data parallelism**, where each process works on a different portion of the training data while maintaining a copy of the model. After computation, the processes communicate their gradients and synchronize the model updates.

6. Benefits of Parallel Execution

Parallel execution allows multiple batches to be processed simultaneously. This can significantly reduce training time and allows large datasets and models to be processed more efficiently. It also enables computational workloads to be distributed across multiple machines.

7. Communication and Scalability Challenges

As the number of processes increases, the amount of communication and synchronization can also increase. Frequent gradient exchange can become a bottleneck, particularly when network bandwidth is limited or communication latency is high. If communication takes too much time compared with computation, adding more processes may provide diminishing performance benefits.

Conclusion

Distributed matrix computation and MPI collective communication provide an important foundation for **large-scale deep learning**. By distributing data and computation among multiple processes and efficiently synchronizing their results, large models can be trained more efficiently. However, achieving good scalability requires careful management of **data distribution, communication, synchronization, and workload balance**.

8. Discussion Questions

1. What is collective communication?

Collective communication is a type of MPI communication in which a group of processes in a communicator participate in the same communication operation. Unlike point-to-point communication, which usually involves two processes, collective communication involves multiple processes.

Examples include `MPI_Bcast()`, `MPI_Scatter()`, `MPI_Gather()`, `MPI_Reduce()`, and `MPI_Barrier()`.

2. How does MPI_Bcast() work?

`MPI_Bcast()` broadcasts the same data from one root process to all other processes in a communicator.

The root process contains the original data, and all other processes receive a copy of that data. Every process participating in the broadcast must call `MPI_Bcast()`.

For example, if process 0 has a matrix or a value required by all processes, it can use `MPI_Bcast()` to distribute it to the other processes.

3. Differentiate MPI_Scatter() and MPI_Gather().

MPI_Scatter()	MPI_Gather()
Distributes data from one root process to all processes.	Collects data from all processes to one root process.
Communication direction is root → all processes.	Communication direction is all processes → root.
Each process receives a portion of the data.	The root receives a portion of data from each process.

Useful for distributing work.	Useful for collecting results.

For example, in matrix processing, `MPI_Scatter()` can distribute different rows of a matrix, while `MPI_Gather()` can collect the processed rows back at the root.

4. Differentiate MPI_Gather() and MPI_Reduce().

`MPI_Gather()` collects the individual data produced by all processes and stores it at the root process. It does not combine or perform any mathematical operation on the data.

`MPI_Reduce()`, on the other hand, combines the values from all processes using an operation such as `MPI_SUM`, `MPI_MAX`, `MPI_MIN`, or `MPI_PROD`, and normally stores the final result at the root process.

For example, if processes produce values 10, 20, and 30:

- `MPI_Gather()` → root receives **10, 20, 30**
- `MPI_Reduce(MPI_SUM)` → root receives **60**

5. Why is MPI_Bcast() useful in matrix multiplication?

In matrix multiplication, multiple processes may need access to the same matrix or other common input data. Instead of sending the same data separately to every process using multiple point-to-point communications, `MPI_Bcast()` can distribute it to all processes efficiently.

For example, if every process needs matrix B to calculate its assigned rows of matrix C, the root process can broadcast matrix B using `MPI_Bcast()`.

This simplifies communication and ensures that all processes have the required common data.

6. Why is row-wise decomposition suitable for matrix-vector multiplication?

In row-wise decomposition, different rows of the matrix are assigned to different processes.

For matrix-vector multiplication: [y = Ax]

each output element (y_i) depends mainly on one row of matrix (A) and the complete vector (x). Therefore, a process can calculate the output values corresponding to its assigned rows independently once it has the vector.

This makes row-wise decomposition simple and suitable because:

- Work can be divided easily among processes.
- Each process handles independent rows.
- Communication is relatively simple.
- The complete vector can be broadcast to all processes.

7. What are the limitations of row-wise decomposition?

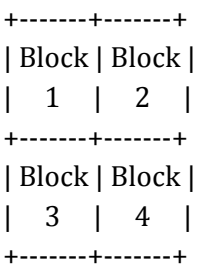
Some limitations of row-wise decomposition are:

1. **Load imbalance:** If the number of rows is not evenly divisible by the number of processes, some processes may receive more rows.
2. **Communication overhead:** The complete vector may need to be distributed to all processes.
3. **Memory requirements:** Each process may need to store the complete vector.
4. **Limited data locality:** Processes work with entire rows, but some operations may benefit from smaller blocks of data.
5. **Scalability limitations:** Increasing the number of processes does not always improve performance because communication and synchronization overhead can increase.

8. What is block decomposition?

Block decomposition divides a large matrix into smaller rectangular blocks or submatrices and assigns these blocks to different processes.

For example, instead of assigning complete rows to processes, a matrix can be divided into a grid of blocks:



Each process can work on one or more blocks.

9. Why can block decomposition improve data locality?

Block decomposition keeps related data elements close together in memory. When a process works on a block, it repeatedly accesses nearby elements instead of jumping across widely separated memory locations.

This improves **data locality**, which can make better use of CPU caches and reduce unnecessary memory access.

It can therefore improve performance for large matrix computations, particularly when the computation repeatedly operates on submatrices.

10. What is the purpose of MPI_Scatterv()?

`MPI_Scatterv()` is a more flexible version of `MPI_Scatter()`.

While `MPI_Scatter()` normally sends the same amount of data to every process, `MPI_Scatterv()` allows the root process to send **different amounts of data to different processes**.

It is useful when the data cannot be divided equally among processes.

For example, if 10 rows must be distributed among 3 processes, the rows could be divided as 4, 3, and 3. `MPI_Scatterv()` can handle this unequal distribution.

11. What happens when the number of data elements is not evenly divisible by the number of Processes?

When the number of elements cannot be divided equally among the processes, an equal distribution is not possible.

For example, if there are 10 rows and 3 processes, they could receive:

- Process 0 → 4 rows
- Process 1 → 3 rows
- Process 2 → 3 rows

The extra elements are distributed among some processes.

Functions such as `MPI_Scatterv()` can be used to explicitly handle these different amounts of data. The goal is to keep the workload as balanced as possible.

12. Why does execution time sometimes stop decreasing as the number of processes increases?

Execution time may stop decreasing because adding more processes also increases the overhead associated with parallel execution.

Important factors include:

- Communication between processes
- Synchronization
- Load imbalance
- Process creation and management overhead
- Limited memory bandwidth
- Network latency
- A portion of the program that cannot be parallelized

After a certain point, the additional processes may spend more time communicating or waiting than performing useful computation.

13. How does communication overhead affect MPI scalability?

Communication overhead can limit the scalability of an MPI program.

As the number of processes increases, processes may need to exchange more data and synchronize more frequently. If communication takes a significant amount of time compared with computation, the benefit of adding more processes decreases.

Therefore, a well-designed MPI program should minimize unnecessary communication, reduce the number of messages where possible, and maintain good workload balance.

14. How does MPI differ from OpenMP in terms of memory architecture?

MPI and OpenMP are primarily designed for different memory models.

MPI	OpenMP
Primarily follows a distributed-memory model.	Primarily follows a shared-memory model.
Processes generally have separate memory spaces.	Threads share the memory of the same process.
Processes communicate using message passing.	Threads communicate through shared variables/memory.
Suitable for multiple machines or nodes.	Commonly used within a single shared-memory system.
Uses MPI processes and ranks.	Uses threads.

MPI can also be combined with OpenMP in **hybrid parallel programming**, where MPI is used between nodes and OpenMP is used within each node.

15. For a very large matrix, what factors should be considered when selecting a decomposition strategy?

For a very large matrix, several factors should be considered:

1. **Matrix size:** Larger matrices may benefit from block-based decomposition.
2. **Number of processes:** The decomposition should provide enough work for each process.
3. **Load balancing:** Work should be distributed as evenly as possible.
4. **Communication cost:** The amount and frequency of data exchanged between processes should be minimized.
5. **Data locality:** The decomposition should allow efficient use of CPU caches and memory.
6. **Memory availability:** Each process should have enough memory for its assigned data and intermediate results.
7. **Network performance:** On an HPC cluster, network latency and bandwidth affect communication.
8. **Nature of computation:** The dependencies between matrix elements should be considered.
9. **Scalability:** The strategy should continue to perform efficiently as the number of processes increases.

For very large matrices, block decomposition can be useful because it can provide good workload distribution and data locality, although the final choice depends on the computation and communication pattern.

9. Reflection

Answer the following in your own words:
1. What was the most important concept you learned from this assignment?
The most important concept I learned was how MPI processes communicate and divide computational work in a distributed-memory system. I understood that processes have separate memory spaces and therefore need communication mechanisms such as point-to-point and collective operations to exchange data and coordinate their work.

2. Which MPI collective operation did you find most useful and why?

I found `MPI_Scatter()` particularly useful because it directly supports data distribution. It allows a large dataset to be divided into smaller portions and assigned to different processes, which is important when parallelizing matrix computations.

`MPI_Bcast()` is also useful when all processes need access to the same data, while `MPI_Gather()` is useful for collecting the results.

3. How is MPI matrix multiplication different from OpenMP matrix multiplication?

MPI matrix multiplication uses multiple processes that generally have separate memory spaces. Therefore, the matrices or portions of matrices have to be distributed among processes, and communication is required to exchange data and collect results.

OpenMP matrix multiplication uses multiple threads within a shared-memory system. The threads can directly access shared matrices, so explicit message passing is generally not required.

Therefore, MPI involves **processes, distributed memory, and message passing**, whereas OpenMP involves **threads and shared memory**.

4. What happens when more MPI processes are added?

When more MPI processes are added, the computational workload can be divided among more processes, which can reduce execution time initially.

However, more processes also mean increased communication, synchronization, and management overhead. If the workload becomes too small per process, or communication becomes a major part of the execution time, adding more processes may provide little or no performance improvement.

5. Did increasing the number of processes always improve performance? Explain using your observations.

No, increasing the number of processes did not necessarily improve performance in every case.

Initially, increasing the number of processes can reduce execution time because more processes work on the computation simultaneously. However, after a certain point, communication, synchronization, and other overheads become more significant.

Therefore, the execution time may stop decreasing or may even increase when too many processes are used. This shows that the number of processes must be selected according to the problem size and the available system resources.

6. Which decomposition strategy would you select for a large matrix and why?

For a large matrix, I would consider **block decomposition** because it divides the matrix into smaller rectangular blocks and can provide good workload distribution and data locality.

Working on smaller blocks can also improve cache utilization because related data is kept close together in memory. However, the final choice would depend on the matrix operation, communication requirements, number of processes, and system architecture.

7. What communication bottlenecks did you observe?

The main communication bottlenecks were the time required to distribute input data, exchange information between processes, synchronize processes, and collect the final results.

As the number of processes increases, these communication operations can become a larger part of the total execution time. For large data, network bandwidth and latency can also affect performance significantly.

8. How could the implementation be further optimized for an HPC cluster?

The implementation could be optimized in several ways:

- Use an appropriate decomposition strategy to maintain workload balance.
- Minimize unnecessary communication between processes.
- Use collective operations efficiently.
- Reduce the number of synchronization points.
- Use non-blocking MPI operations where appropriate to overlap communication with computation.
- Improve data locality through block decomposition.
- Select an appropriate number of MPI processes based on the matrix size and cluster resources.
- Use hybrid MPI + OpenMP parallelism when suitable.
- Benchmark different process counts to identify the point where performance is optimal.
- Use optimized numerical libraries such as BLAS-based or other HPC matrix computation libraries when applicable.

These optimizations can reduce communication overhead and improve the overall scalability and performance of the application on an HPC cluster.

10. Deliverables
Submit the following:
A. Programs
1. MPI Broadcast
2. MPI Scatter/Gather
3. MPI Reduce
4. MPI Matrix–Vector Multiplication
5. MPI Matrix–Matrix Multiplication
6. Decomposition comparison
7. MPI_Scatterv() implementation

B. Documentation

Include:

1. Aim and objective
2. Algorithm
3. Source code
4. Sample input/output
5. Performance tables
6. Graphs
7. Speedup calculation
8. Efficiency calculation
9. Analysis of results
10. AI/HPC case study
11. Discussion-question answers
12. Reflection
13. Conclusion

C. Performance Evidence

Include execution results for different:

1. Matrix sizes
2. Number of MPI processes
3. Decomposition strategies

11. Expected Learning Outcomes

After completing this assignment, you should be able to:

1. Implement MPI collective communication operations such as broadcast, scatter, gather and reduction.
2. Apply MPI to matrix-vector and matrix-matrix computations.
3. Analyze different data decomposition strategies for distributed-memory systems.
4. Measure execution time, speedup, efficiency and communication overhead.
5. Evaluate the effect of process count and data distribution on MPI performance.
6. Relate MPI-based parallel computation to real-world HPC and AI applications.